INFO5995 Assignment 2 — Part A:Network Security Analysis of a2_case1.apk

XIANGSHENBAO The University of Sydney
YISHENGLIN The University of Sydney
YANGQU The University of Sydney
WEIFENGWU The University of Sydney
HAOXUANYANG The University of Sydney
YIFAN YANG The University of Sydney

Abstract

We analyse a2_case1.apk (package com.example.mastg_test0019), an Android WebView app that loads a remote page. Static analysis shows three insecure-transport defects: (E1) cleartext HTTP is permitted in the manifest, (E2) the loaded URL uses http://, and (E3) the custom WebViewClient accepts every TLS error by calling handler.proceed(). Each defect can break channel confidentiality and integrity against a same-network attacker. We provide code-level evidence (Figs. 2–3) and recommend a Network Security Config-based fix.

1. Decompilation & App Overview

We unpacked the APK with unzip and analysed it with Androguard 4.1.3 and jadx-gui. The app has one Activity: minSdk 19, targetSdk 34, and android.permission.INTERNET is the only dangerous-class permission. MainActivity.onCreate() inflates a WebView, sets a custom WebViewClient, and calls loadUrl(...). There is no other network code. The package name mastg_test0019 matches OWASP MASTG test case 0019 (insecure network communication) [1].

2. System & Threat Model

Fig. 1 shows the data flow. The trusted side is the user's device and the origin server. Everything between them — Wi-Fi AP, LAN, ISP, upstream routers — is untrusted. The attacker we consider is on the same network segment as the user (peer on a public Wi-Fi, rogue AP, or someone running ARP/DHCP/DNS spoofing). They need no credentials, no malware on the device, and no physical access.

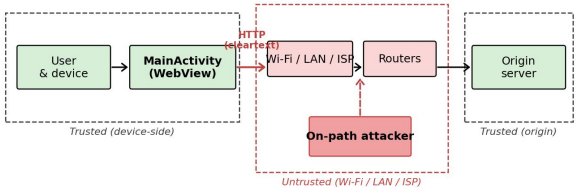

Fig. 1: System and threat model. The attacker sits on the untrusted network segment between app and server.

Protected assets. A1 — integrity of the HTML/JS rendered in the WebView. A2 — confidentiality of the request URL, headers, cookies, and any query parameters. A3 — the in-app execution context, because WebView runs returned JavaScript with the app's identity.

Attacker capabilities. C1 — read traffic in plaintext. C2 — modify responses in flight (e.g. inject a <script>). C3 — present a forged certificate if the URL ever uses HTTPS, because the app accepts TLS errors (§3.3). C4 — redirect requests to a different host via DNS or ARP spoofing.

3. Vulnerability: Insecure Transport

The app contains three independent transport-layer defects (E1–E3) in the manifest and in MainActivity. Figs. 2–3 show the decompilation evidence and Fig. 4 traces an end-to-end MITM run that combines them.

3.1 E1: Cleartext traffic globally permitted

The <application> element sets usesCleartextTraffic="true" and does not declare android:networkSecurityConfig, so plain HTTP to any host is allowed (Fig. 2):

```
<application ...
  android:usesCleartextTraffic="true" ...>
```

```
<application
    android:theme="@style/Theme.MASTGTEST0019"
    android:label="@string/app_name"
    android:icon="@mipmap/ic_launcher"
    android:allowBackup="false"
    android:supportsRtl="true"
    android:extractNativeLibs="true"
    android:fullBackupContent="@xml/backup_rules"
    android:usesCleartextTraffic="true"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:appComponentFactory="androidx.core.app.CoreComponentFactory"
    android:dataExtractionRules="@xml/data_extraction_rules">
    <activity
        android:name="com.example.mastg_test0019.MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN"/>
            <category android:name="android.intent.category.LAUNCHER"/>
        </intent-filter>
    </activity>
```

Fig. 2: AndroidManifest.xml — `usesCleartextTraffic="true"` on the `<application>` element. No `networkSecurityConfig` is referenced.

On Android 9+ (API 28), the platform default is to deny cleartext. Setting this attribute to true is an explicit opt-out.

3.2 E2 + E3: Cleartext URL and unconditional TLS-error accept

Fig. 3 shows the body of MainActivity.onCreate(). Three things are wrong in this single method:

- **E2** — webView.loadUrl("http://www.example.com") issues the request over plain HTTP.
- **E3** — the inner class MainActivity$1 (a WebViewClient) overrides onReceivedSslError and calls sslErrorHandler.proceed() with no checks. Every TLS error — expired, self-signed, hostname-mismatched, untrusted CA — is accepted.
- Auxiliary — MainActivity$2 implements HostnameVerifier.verify() and returns true for any input, removing hostname validation.

```
    }
});
WebView webView = (WebView) findViewById(R.id.webview);
webView.setWebViewClient(new WebViewClient() { // from class: com.example.mastg_test0019.MainActivity.1
    @Override // android.webkit.WebViewClient
    public void onReceivedSslError(WebView webView2, SslErrorHandler sslErrorHandler, SslError sslError) {
        sslErrorHandler.proceed();
    }
});
webView.loadUrl("http://www.example.com");
new HostnameVerifier() { // from class: com.example.mastg_test0019.MainActivity.2
    @Override // javax.net.ssl.HostnameVerifier
    public boolean verify(String str, SSLSession sSLSession) {
        return true;
    }
};
```

Fig. 3: `MainActivity.onCreate()` (jadx output). The same method contains the cleartext `loadUrl`, the SSL-error override that calls `proceed()`, and the always-true `HostnameVerifier`.

E2 alone makes the request readable and modifiable. E3 means the app stays exploitable even if E1 and E2 are later fixed and the URL is switched to https://: the attacker just presents a self-signed certificate and the app accepts it.

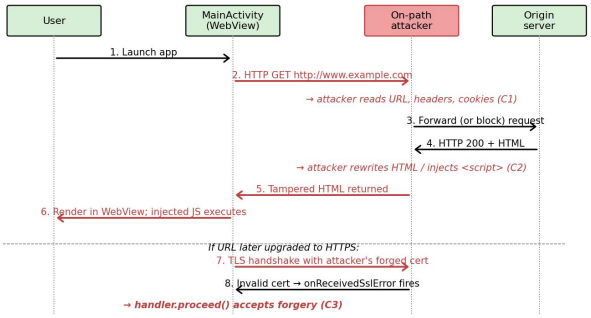

Fig. 4: End-to-end MITM trace. Steps 1–6 use the cleartext path (E1+E2). Steps 7–8 show the HTTPS fallback case (E3).

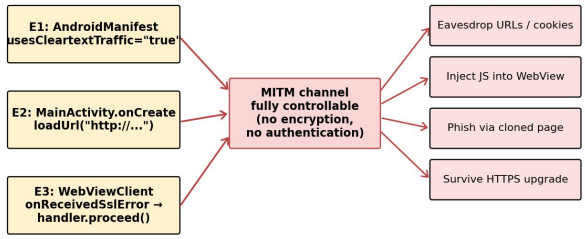

Fig. 5: How E1, E2, and E3 combine into a single attack surface and the four impact categories that follow.

3.3 Impact

Combining the attacker capabilities (C1–C4) with the defects (E1–E3, Fig. 5), a same-Wi-Fi attacker can:

- Read the full request URL, headers, cookies, and any query parameters (loss of A2).
- Rewrite the HTML body and inject JavaScript that runs inside the WebView (loss of A1). If addJavascriptInterface is added later, this becomes code execution against A3.
- Serve a phishing page that looks identical to the real origin and harvest credentials.
- Keep the attack working after a future http→https migration, because E3 disables certificate-based authentication.

We estimate CVSS v3.1 AV:A/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:N = **7.4 (High)**: adjacent network, low complexity, no privilege, user only needs to launch the app; both confidentiality and integrity are fully compromised.

4. Mitigation

All three defects share the same root cause: the app overrides Android's secure-by-default transport behaviour, so the fix should restore those defaults.

M1 — Forbid cleartext via Network Security Config (fixes E1) [2]. Remove usesCleartextTraffic="true" from the manifest and add android:networkSecurityConfig="@xml/network_security_config" with:

```
<network-security-config>
  <base-config cleartextTrafficPermitted="false">
    <trust-anchors>
      <certificates src="system"/>
    </trust-anchors>
  </base-config>
</network-security-config>
```

Cleartext is then refused by the platform, so any stray http:// URL fails closed.

M2 — Use HTTPS at the call site (fixes E2). Change loadUrl("http://www.example.com") to the https:// URL. Together with M1, regressions to cleartext are caught.

M3 — Remove the SSL-error override (fixes E3). The default WebViewClient cancels requests with invalid certificates, which is correct because invalid certificates indicate either server misconfiguration or an active attacker. If non-default behaviour is needed for an internal staging host, gate it on the host *and* a pinned key — never call proceed() unconditionally. Also remove the HostnameVerifier override so the platform's OkHostnameVerifier is used.

M4 — Defence in depth: certificate pinning. Pin the origin's SPKI in the same NSC via <pin-set>. Pin the origin's SPKI in the same NSC via <pin-set>, narrowing trust from the full system CA store to one key.

Why these reduce risk. M1 closes the cleartext escape hatch. M2 provides encryption and integrity. M3 restores certificate authentication, without which encryption is insufficient against active MITM. M4 limits public-CA compromise impact. After these fixes, C1 is defeated by encryption, C2 by TLS integrity, and C3 by certificate validation plus pinning.

We used ChatGPT for report structure, wording, formatting, and requirement checking. All AI-assisted outputs were reviewed and edited by the group.

References.

[1] OWASP Foundation.
Mobile Application Security Testing Guide (MASTG): Test Case 0019.OWASP Mobile Application Security, 2025. https://mas.owasp.org/MASTG/.

[2] Android Developers.
Network Security Configuration.Android Developers Documentation, 2025. https://developer.android.com/privacy-and-security/security-config.